

Latency- and Privacy-Aware Convolutional Neural Network Distributed Inference for Reliable Artificial Intelligence Systems

Yuhao Hu , Xiaolong Xu , Senior Member, IEEE, Lianyong Qi , Senior Member, IEEE, Xiaokang Zhou , Member, IEEE, and Xiaoyu Xia 

Abstract—Reliable artificial intelligence (AI) systems not only propose a challenge on providing intelligent services with high quality for customers but also require customers' privacy to be protected as much as possible during process of the services. Given the ultrahigh computing load brought by deep-learning-based intelligent services to edge devices and the ultralong distance between edge and cloud, low-latency requirement of intelligent services is hard to meet in single edge computing or cloud computing. Edge-cloud collaborative inference of deep neural networks (DNNs) is considered a feasible solution to the problem. However, former work has not reduced the inference latency to the greatest extent and has not considered privacy protection in distributed systems. To solve the problem, we first establish a novel queue mechanism. Then, the convolution layer split decisions are made based on deep reinforcement learning (DRL) to realize the parallel inference of convolutional neural networks (CNNs) for inference latency reduction. Next, for each CNN, the partition decision is made based on brute force algorithm to further reduce inference latency and protect customers' privacy. Finally, simulation results show that our method performs better than the existing other methods.

Impact Statement—Advance of AI drives the popularization of intelligent services, such as automatic driving and smart healthcare, which bring great convenience to people's lives. However, ultralow service response time and high customers' privacy security are two main challenges in reliable AI systems. The CNN distributed inference method proposed in this article overcomes the issues. It provides a novel queue mechanism to minimum response time of the most urgent services while

reducing waiting time of those with low priorities. Additionally, the method we propose efficiently protects customers' privacy while ensuring the accuracy loss lower than 10%, performing better than traditional difference privacy whose accuracy loss is up to 20%. The proposed method can greatly improve quality of intelligent services (QoSs) while protecting customers' privacy in a reliable AI system.

Index Terms—Convolutional neural network (CNN) parallelism, CNN partition, inference latency, privacy protection, reliable artificial intelligence (AI) system.

I. INTRODUCTION

NOWADAYS, intelligent services based on deep learning (DL) are becoming more popular with the development of artificial intelligence (AI) and communication technology. As one of the main deep neural networks (DNNs), convolutional neural networks (CNNs) are widely used in vision applications such as object detection, object classification, and visual reality [1], [2], [3]. Considering the relatively less computing resources, little storage space and limited power of end devices and edge servers [4], it is difficult to deploy and infer a whole CNN model with large parameter quantity and computation amount completely with high efficiency on an edge device [5], [6]. One traditional solution to the problem is to offload the inference task of the CNN to the cloud server. With so strong computing power, the cloud server is able to infer a large CNN model with low latency and improve quality of intelligent services (QoSs).

However, the long distance between edge and cloud causes huge transmission latency of raw input data from intelligent service customers [7]. Too large data transmission latency significantly increases the total response time of intelligent services and reduces the QoS. Furthermore, traditional cloud computing brings large security risk to customers' privacy. Specifically, remote location of the cloud server causes prolonged exposure of customers' privacy data in the process of transmission, which largely increases the possibility for customers' privacy to be stolen. Also, the centralized and outsourcing pattern of cloud computing make customers' data stored on the cloud easier to be visited and attacked by third-party attackers [8]. Distributed inference for CNNs on various devices is an excellent method to improve inference efficiency and reduce inference latency

Manuscript received 5 October 2023; revised 20 December 2023, 12 January 2024, and 5 February 2024; accepted 11 February 2024. Date of publication 19 February 2024; date of current version 27 February 2025. This work was supported in part by the Major Research plan of the National Natural Science Foundation of China under Grant 92267104, in part by the National Science Foundation of Jiangsu Province of China under Grant BK20211284, and in part by the Financial and Science Technology Plan Project of Xinjiang Production and Construction Corps under Grant 2020DB005. This article was recommended for publication by Associate Editor Qinghua Lu upon evaluation of the reviewers' comments. (Corresponding authors: Xiaolong Xu; Xiaokang Zhou.)

Yuhao Hu and Xiaolong Xu are with the School of Software, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: 202212210036@nuist.edu.cn; xlxu@nuist.edu.cn).

Lianyong Qi is with the College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266850, China (e-mail: lianyongqi@gmail.com).

Xiaokang Zhou is with the Faculty of Data Science, Shiga University, Hikone 522-8522, Japan (e-mail: zhou@biwako.shiga-u.ac.jp).

Xiaoyu Xia is with the School of Computing Technologies, RMIT University, Melbourne, VIC 3000, Australia (e-mail: xiaoyu.xia@rmit.edu.au).

Digital Object Identifier 10.1109/TAI.2024.3366880

because it considers the advantages of both edge computing and cloud computing [9]. In detail, existing work has proved that the size of most intermediate output from CNNs is smaller than that of raw input from users [10]. So a CNN model can be partitioned into two or more parts, the shadow ones with lower computing intensity deployed at edge and the deep ones with higher computing intensity deployed on the cloud. This distributed inference pattern is called model partition. In model partition, data transmission latency is reduced for smaller transmission size, and computing intensive layers are still inferred efficiently on the cloud. Besides model partition, there is another distributed inference pattern called model parallelism. In model parallelism, each layer of a DNN is split into several parts, which are inferred on different devices in parallel. Model parallelism breaks through the layer-by-layer inference pattern of model partition, thus further reducing computing latency of each layer. But more data transmission latency may be caused by the increasing interaction between different devices and there may exist inference accuracy loss because of the integrity breach for layers in model parallelism.

Besides response time, privacy protection is also attracting more attention with popularization of intelligent services [11], which facilitates researching on the reliable AI system. In a reliable AI system, the response time of services should be minimized to improve customers' experience while protecting customers' privacy from being attacked and stolen [12], in order to make the services trusted by humans. Depending on the information of DNN models which can be achieved by attackers, there are two kinds of attack during DNN inference process, which are white-box attack and black-box attack [13], [14]. In white-box attack, the parameters of DNN models are known by attackers and they can query inference of DNNs deployed in servers. In black-box attack, attackers can only access the input and output of DNNs deployed in servers without getting the internal parameters. Considering great attention paid by service providers to protecting customers' privacy, white-box attack is rare in reality. Besides, former work mainly studies how to protect customers' privacy without considering a distributed system. So how to effectively protect customers' privacy from black-box attack during distributed inference process is worth studying. It should be emphasized that split and partition are two different concepts in this article, which cannot be confused. In detail, split is an operation cutting a layer into several parts to achieve model parallelism, while partition cuts a complete DNN model into several parts and each layer of the DNN is inferred completely without split.

To overcome the above mentioned challenges, in this article, we design a CNN distributed inference system which joins model partition and model parallelism together. We put each service request in a distributed inference system into a corresponding queue according to its priority. Then, we split each convolution layer (conv) into several parts which are inferred in parallel on different edge servers for inference latency reduction. At last, we find a proper partition point to achieve edge-cloud collaborative CNN inference for further latency reduction while decreasing the possibility for customers' privacy to be stolen. Compared to former work, two challenges are tackled in

this article. First, the inference latency of CNN can be further reduced. Second, privacy protection should be considered during distributed inference process of CNN. The main contributions of this article are as follows.

- 1) An optimized queue mechanism is designed in this article to ensure the minimal response time of services with the highest priorities while reducing waiting time of services with relatively low priorities as much as possible.
- 2) A deep reinforcement learning (DRL)-based CNN parallelism method is proposed to shorten CNN inference latency.
- 3) A brute force algorithm-based CNN partition method is proposed to further decrease CNN inference latency while protecting customers' privacy.
- 4) The effectiveness of the proposed method is confirmed by the comparative experiments.

The rest of this article is organized as follows. The related work is summarized in Section II. We describe the system model and formulate the problem in Section III. The design of our CNN distributed inference system is described in Section IV, followed by results of simulation in Section V. Section VI concludes our work and elaborates the future work.

II. RELATED WORK

In this section, the existing researches related to our work are reviewed from the aspect of distributed inference of DNNs and privacy protection in reliable AI systems.

Devices with limited computing power at edge are hard to infer a complete DNN model in a short time for the intensive computation of DNNs. On the other hand, the long distance between edge and cloud causes that it may take long time to transmit input data from customers to the cloud server in cloud computing. Distributed inference for DNNs becomes a promising scheme for low-latency DNN inference. In terms of model partition, Hu et al. [15] proposed an adaptive DNN partition approach based on Max-Flow/Min-Cut algorithm [16] to minimize the end-to-end latency of DNN inference. This partition method dynamically adjusts the DNN partition strategy in real time according to the change of network bandwidth and is appropriate for DNN models not only with chain topological structure but also with directed acyclic graph (DAG) topological structure. Considering that an end device often simultaneously processes several tasks, Duan and Wu [17] designed a framework which jointly optimizes DNN partition and scheduling based on convex optimization theory and Johnson's rule [18]. The time taken to complete all tasks simultaneously processed on the end device is minimized through the framework. Li et al. [19] proposed a complete framework to jointly optimize DNN partition, target server selection and computing resource allocation based on Max-Flow/Min-Cut algorithm and generalized assignment problem in a scenario with multiple end devices and edge servers. The framework maximizes the throughput of each edge server while ensuring the largest required response time of service from each customer. In terms of model parallelism, Kim et al. [20] split each layer of a DNN into several parts according to the different classes of features in input. Different

parts of the original layer are deployed and inferred in parallel on different servers to reduce inference latency of the DNN. Based on the work of Kim et al., and considering the possible accuracy loss caused by model parallelism and different response latency requirement of various intelligent services, Tuli et al. [21] proposed a novel DNN distributed inference policy to adaptively decide proper pattern of distributed inference (i.e., model partition and model parallelism) for each service based on multiarmed-bandits model and select target server for partial offloading of each service based on DRL. Different from the model parallelism method mentioned above, Zeng et al. [22] designed a system that deploys the complete CNN model on all devices and splits the input data according to the heterogeneous computing capabilities of different devices and constantly changing network conditions. This system effectively reduces latency and energy consumption of CNN inference.

Protecting customers' privacy from being stolen by attackers is always a significant study point in the field of reliable AI system. Kairouz et al. [23] proposed an end-to-end system to achieve security aggregation for the problem of how to ensure the security of the results and the privacy of customers when the distributed training results are aggregated. The system adds discrete Gaussian noise before aggregation and conducts privacy analysis on it. It has been proved that this system can be well applied to federated learning to achieve privacy protection functions. Zhao et al. [24] proposed a secure and efficient aggregation framework to solve the common Byzantine problem in distributed training, which uses trusted hardware Intel SGX to provide privacy assurance and aggregation efficiency and aggregates a large number of model effects in a trusted execution environment through two different data storage modes. Xie et al. [25] proposed a cloud-based DNN inference framework to jointly optimize privacy preserving and inference efficiency. They utilized a novel matrix permutation and parallelization to improve the computational efficiency of linear layers and provided privacy guarantee through homomorphic encryption. Tian et al. [26] proposed an edge-assisted framework to enable low-latency inference of DNNs while achieving privacy protection. They offloaded the most computing intensive layers to edge servers and used novel encryption to ensure that the DNN is inferred in real time over encrypted data.

However, to the best of our knowledge, there has not been any work that combines model parallelism with model partition together and comprehensively utilizes their advantages. In addition, few studies have focused on customers' privacy protection in DNN distributed inference systems. To this end, we propose a CNN distributed inference method that reduces inference latency as much as possible jointly through CNN parallelism and partition while protecting customers' privacy.

III. SYSTEM MODEL AND PROBLEM DEFINITION

In this section, the system model of latency- and privacy-aware distributed inference for CNNs in a reliable AI system is proposed. First, the system framework for CNN distributed inference, the process of conv split and result aggregation, and the novel queue mechanism are presented. Next, the threat model of

TABLE I
TABLE OF MAIN SYMBOLS IN THIS ARTICLE

Symbol	Description
U	The set of all end devices.
u_n	The n th end device.
E	The set of all edge servers.
e_m	The m th edge server.
N	The number of end devices.
M	The number of edge servers.
p_1, p_2	Controller 1, controller 2.
c	The cloud server.
$r_{m,j}$	The split rate of the j th conv for the m th edge server.
x_j	FLOPs of the j th conv.
$k_{m,j}$	The number of computing resources allocated by the m th edge server to infer the j th conv.
η_m	FLOPs of per thread on the m th server.
$t_{m,p}$	The transmission time between controller 1 and the m th edge server.
$x_{m,j}^{out}$	The size of output of the j th conv from the m th edge server.
$s_{m,p}$	The data transmission speed between the m th edge server and controller 1.
t_e	The total inference latency of all convs in a CNN model inferred at edge.
q	The index of the conv which partitions a CNN.
t_c	The total inference latency on the cloud.
$t_{c,p}$	The data transmission speed between controller 1 and the cloud server.
η_c	The computation power of the cloud.
$s_{c,p}$	The data transmission speed between controller 1 and the cloud server.
$x_{q,c}^{out}$	The size of transmitted data between controller 1 and the cloud server.
A	The number of convs in a CNN.
Q	The total number of layers in a CNN.

black-box attack is established. Then, the inference latency of CNNs on edge servers and cloud as well as the transmission latency of intermediate outputs between different devices is modeled. Finally, the CNN split problem is formulated as a minimum optimization problem to minimize the total inference latency of all intelligent services while protecting customers' privacy as much as possible. Key notations in this article are listed in Table I.

A. System Framework

As shown in Fig. 1, the edge-cloud collaboration-based CNN distributed inference system consists of several intelligent end devices, several edge servers, two controllers, and a cloud server.

We assume that there are N intelligent service customers and M edge servers. Each intelligent service customer has an end device, from which a request of the service based on a certain type of CNN is produced. All types of CNNs possible to be used have been deployed on edge servers and the cloud server in advance. Each service request is transformed into a CNN inference task to be processed and is given a priority in controller p_1 . Considering various priorities of tasks from different end devices and to shorten the service response latency of tasks with high priorities as much as possible while reducing waiting time of tasks with low priorities, a novel queue mechanism is designed, which is described in Section III-C. In this section, the distributed inference process of a single task is presented.

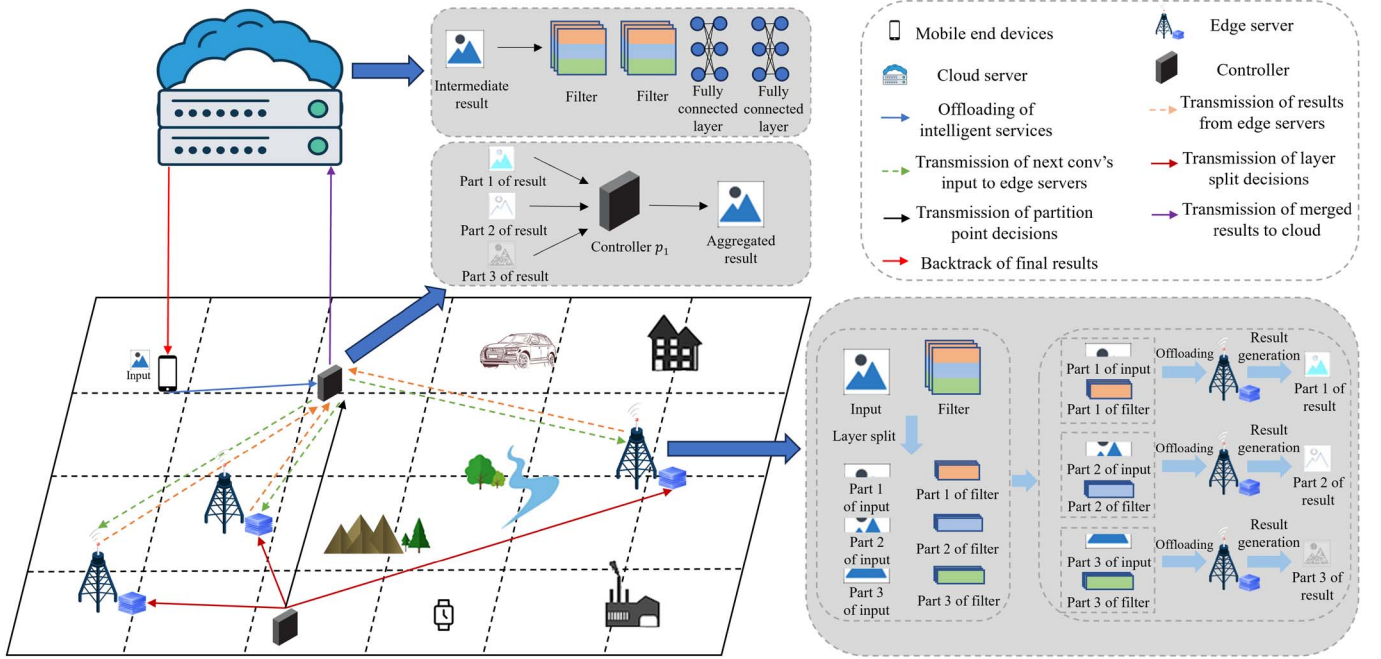


Fig. 1. CNN distributed inference framework for reliable AI systems.

On the whole, the distributed inference of each task is divided into two stages, which are parallel inference at edge and inference without parallelism on cloud. Specifically, after detecting the network fluctuation at edge or between edge and cloud, controller p_2 first dynamically adjusts the model parallelism decision according to the edge network bandwidth, during which each conv of a CNN is split into several parts and each part can be inferred on different edge servers in parallel. Controller p_2 then finds the partition point of the CNN according to the edge-cloud network bandwidth and the security of output produced by each conv. The partition point partitions the whole CNN model into two parts, i.e., the shadow one inferred in parallel at edge and the deep one completely inferred on the cloud. The real-time model parallelism decision and model partition decision are, respectively, sent to edge servers and controller p_1 . Let $U = \{u_1, u_2, \dots, u_N\}$ denote the set of end devices. In terms of CNN distributed inference, end device u_n generates a service request (i.e., a CNN inference request) and sends raw input to controller p_1 . Controller p_1 puts the task transformed from that service request into a queue according to the priority. Then, input from end device u_n is sent to each edge device by controller p_1 . On each edge server, according to the corresponding part of the first conv inferred on it, the needed part of the original input is extracted and processed. It is worth noting that the parallel inference processes of one conv on different edge servers are independent of each other. Let $E = \{e_1, e_2, \dots, e_M\}$ denote the set of edge servers. Inference result from each edge server is returned to controller p_1 and aggregated into a complete output in controller p_1 . Next, the aggregated result is sent to each server for parallel inference and the output from different edge servers is aggregated in controller p_1 again. This process repeats until the parallel inference of all convs decided to be inferred at edge

is finished. Afterward, the aggregated result is transmitted to the cloud c and inferred in complete without split and parallelism. Considering the tiny inference latency of other feature extraction layers (e.g., pooling layers) next [27] compared with convs, we infer them in controller p_1 completely. Additionally, considering the intensive computation of fully connection layers, they are only completely inferred on the cloud [28]. Convs are either inferred on edge servers in parallel or completely inferred on the cloud, which depends on the decision of model partition. We assume that connection between edge servers and controllers, as well as connection between the controllers, is achieved through single-mode fiber (SMF) [29]. Connection between controller p_1 and the cloud c is achieved by WiFi [30].

B. Conv Split and Result Aggregation

The detailed process of conv split and intermediate result aggregation is shown in Fig. 2. As presented in the first part of Fig. 2, convs are split along their heights. After the split of a conv, each obtained part of the original conv only convolves with a part of the input. For a certain part of the original conv, the corresponding convolved part of original input is gotten as follows:

$$h_1 = y \quad (1)$$

$$h_2 = z - (v - w) \quad (2)$$

where h_1 and h_2 are start and end locations to get the convolved part in the original input. y and w are start and end locations of a split part in the original conv. z and v are heights of the original input and conv, respectively.

In the second part of Fig. 2, different parts of the original conv are inferred in parallel on different edge servers, and each edge server produces an intermediate result.

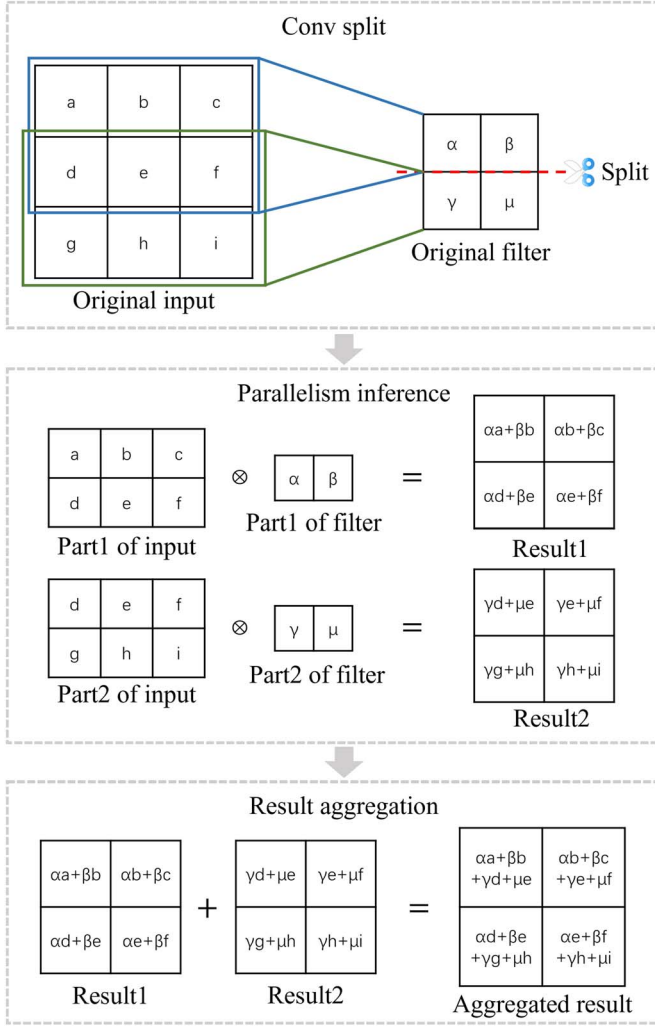


Fig. 2. Process of conv split and result aggregation.

The last part of Fig. 2 shows that the intermediate results from different edge servers are aggregated through simple addition operations. It is easy to prove that the aggregated result is exactly the same as the result obtained through normal inference without conv split and result aggregation.

C. Novel Queue Mechanism

We assume that there are only a few kinds of CNNs possible to be used in a distributed inference system, and services based on the same kind of CNN are sequentially and alternately processed using a novel queue mechanism. The workflow of the novel queue mechanism is shown in Fig. 3. First, the allocation of computing resources for each service request in each edge server is presented in the left part of Fig. 3. Specifically, the process of resource allocation consists of two steps. In step one, computing resources in each edge server are grouped for each kind of CNN according to its proportion of floating-point operations (FLOPs) among all CNN models. In step two, in terms of the computing resources allocated to a certain kind of CNN, they are further grouped for each conv of the CNN

according to the proportion of FLOPs of each conv among the CNN. Note that the allocation of computing resources is conducted offline and the resource allocation decisions will not change in the future. Besides, the number of resources allocated to each conv of a certain kind of CNN will not be used to infer other convs. After resource allocation, the details of sequential and alternate task process are shown in the right part of Fig. 3. For ease of understanding, we explain this process using an example. We assume that there are three service requests with different priorities, which reflect the latency sensitive of different tasks, based on the same kind of CNN, and they are successively put into a queue according to their priorities. In the beginning, the various parts of the first conv (conv1) of the task with the highest priority (task1) are inferred in parallel on different edge servers. When the inference of conv1 of task1 is finished, the second conv (conv2) of task1 begins to be inferred. Meanwhile, the conv1 of task with the middling priority (task2) is also inferred. Next, conv1 of task with the lowest priority (task3), conv2 of tasks and the third conv (conv3) of task1 are simultaneously inferred on the edge servers in parallel. For each task, the process of parallel conv inference continues until the convs, decided to be inferred at edge, of that task are all inferred, and the aggregated intermediate feature map will be sent to the cloud for further inference. It is obvious that from the macroscopic angle, tasks with different priorities are inferred one by one, which conforms to the characteristic of sequential task process. From the microcosmic angle, although all tasks in a queue are not able to be processed simultaneously in an absolute sense, the j th conv of the n th task and the $(j+1)$ th conv of the $(n+1)$ th task can be inferred at the same time. This characteristic of “not completely simultaneous process of multiple tasks” is named as “alternate process of tasks” in this article. In addition, it should be emphasized that different queues are independent of each other and tasks in different queues are processed simultaneously.

D. Threat Model of Black-Box Attack

Facing the responsible AI system, we consider the threat model of black-box attack in cloud-edge collaboration-based CNN distributed inference system in this section. As mentioned above, the whole CNN is partitioned into two parts, one inferred in parallel at edge while the other inferred completely on the cloud, which are, respectively, denoted as f_{ℓ_1} and f_{ℓ_2} . We assume that devices at edge are trusted and privacy-reliable, while the cloud is untrustworthy and the input to the cloud may be stolen [13], [31]. Compared to white-box attack, attackers can only query the part of CNN model deployed at edge without getting the internal parameters in black-box attack. In this case, attackers need to train an inverse network g_{ℓ} . After getting g_{ℓ} , attackers use the intermediate output stolen by them as input of g_{ℓ} and get a substitution of customers' raw input with high similarity. The loss function used to train the inverse network g_{ℓ} is presented as

$$g_{\ell} = \underset{o}{\operatorname{argmin}} \frac{1}{z} \sum_{i=1}^z \operatorname{MSE}(o(b_i), w_i) \quad (3)$$

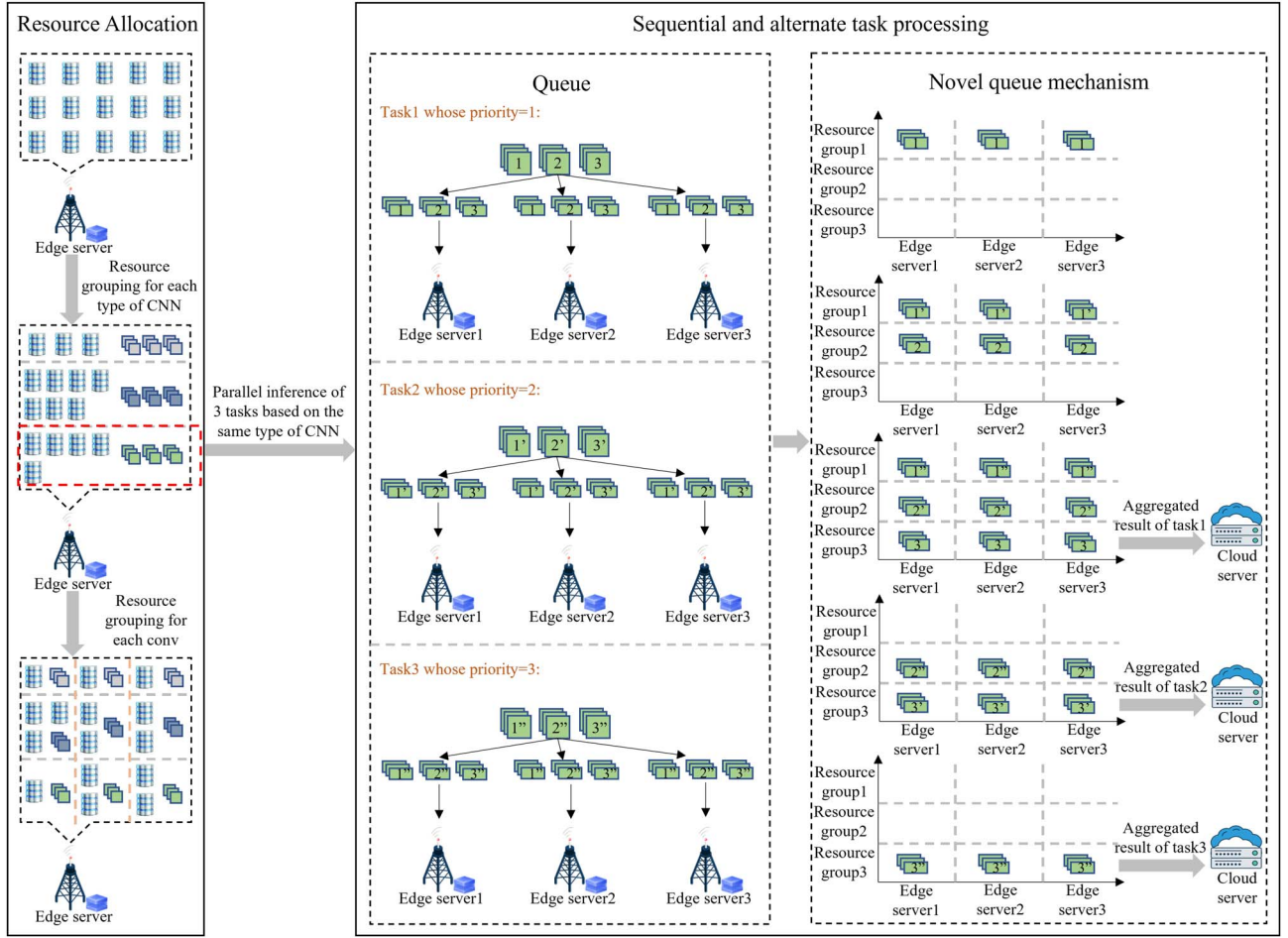


Fig. 3. Novel queue mechanism.

where o is the inverse network to be optimized, g_ℓ is the target inverse network, z is the total number of pixels in samples generated by attackers, b_i is the i th pixel in the input of o , w_i is the i th pixel in the input of $f_{\ell1}$, and MSE represents the mean squared error.

E. Distributed Inference Model

At edge, for each conv, the input is split into several parts in controller p_1 and each part is transmitted to a certain edge server. Each edge server begins the inference process after receiving the corresponding input. Each edge server sends the corresponding intermediate output to controller p_1 instantly when it finishes the inference process, and controller p_1 starts to aggregate the intermediate output from all edge servers once after it receives all output. Note that the output from each edge server has the same size with the result got through normal inference without any conv split and the aggregated result is just the summation of results from all edge servers.

Overlooking the inference latency of other feature extraction layers and the intermediate output aggregating delay, and considering the parallel inference process of convs at edge, the parallel inference latency of the j th conv in a certain CNN

model is presented as

$$t_j = \max \frac{r_{m,j} \cdot x_j}{k_{m,j} \cdot \eta_m} + t_{m,p} \quad (4)$$

where $r_{m,j} \in [0, 1]$ is the split rate of the conv for the m th edge server e_m , and $m \in [1, M]$. x_j is FLOPs of the j th conv. $k_{m,j}$ is the number of computing resources allocated by edge server e_m to infer the conv. η_m denotes computation speed of each unit of computing resource of edge server e_m , which is measured as FLOPs. $t_{m,p}$ denotes the transmission latency between edge server e_m and controller p_1 , which is given by

$$t_{m,p} = \frac{r_{m,j} \cdot x_j + x_{m,j}^{\text{out}}}{s_{m,p}} \quad (5)$$

where $x_{m,j}^{\text{out}}$ is the output of the j th conv from edge server e_m , and $s_{m,p}$ is the data transmission speed between edge server e_m and controller p_1 . Given the inference latency of a single conv, the total inference latency of all convs in the CNN model inferred at edge is presented as

$$t_e = \sum_{j=1}^q t_j \quad (6)$$

where q denotes the index of the conv which is the CNN partition point.

After reaching the CNN partition point, the aggregated intermediate output is transmitted from controller p_1 to the cloud server c for further inference. The total inference latency on the cloud c is presented as

$$t_c = \sum_{i=q}^Q \left(\frac{x_i}{\eta_c} \right) + t_{c,p} \quad (7)$$

where x_i is the FLOPs of the i th layer in the CNN model which is inferred on cloud c , Q is the total number of layers in the CNN model (only consider the number of convs and fully connected layers), η_c is computation power of cloud c which is also represented by FLOPs, and $t_{c,p}$ is the data transmission speed between controller p_1 to the cloud server c . $t_{c,p}$ is given by

$$t_{c,p} = \frac{x_{q,c}^{\text{out}}}{s_{c,p}} \quad (8)$$

where x_q is the size of transmitted data between controller p_1 to the cloud server c , and $s_{c,p}$ is the data transmission speed between controller p_1 to the cloud server c .

F. Problem Formulation

According to what is mentioned above, the total distributed inference latency of a certain CNN model consists of the parallel inference latency at edge and complete inference latency on the cloud. It is given by

$$t_{\text{total}} = t_e + t_c. \quad (9)$$

The optimization goal of this article is to minimize the total response time of all intelligent services in a CNN distributed inference system while protecting the privacy of customers for the realization of a reliable AI system. Considering that the total response time of all intelligent services is optimized through the novel queue mechanism, we study the minimization of inference latency for a single service here. Given the customers' privacy protection, we add SSIM into the final objective function. SSIM is a significant indicator used to evaluate the similarity between two pictures [32]. Here, we use this indicator to assess the quality of data achieved by attackers. The lower the SSIM value, the better customers' privacy is protected. So the final objective function is presented as

$$\min t_{\text{total}} + \alpha \cdot \text{ssim} \quad (10)$$

$$\text{s.t. } m \in [1, M] \quad (11)$$

$$r_{m,j} \in [0, 1] \quad (12)$$

$$\sum_{m=1}^M r_{m,j} = 1 \quad (13)$$

$$q \in [1, A] \quad (14)$$

where α is an adjustment factor to adjust the weight of ssim and A is the number of convs in the CNN model. Constraint (9) means that each conv is inferred by at least one edge server and at most M edge servers if it is inferred in parallel at edge. Constraint (10) means that the split rate of the j th conv should be in the range between 0 and 1. Constraint (11) means that

all parts of the j th conv should be inferred by edge servers. Constraint (12) means that the location of a CNN partition point must be between the first conv and the last conv.

IV. METHOD DESIGN

In this section, we present details on design of the CNN distributed inference system. Specifically, we solve the original multiobjective optimization problem in two steps. First, we find the decision of conv split for model parallelism at edge using an algorithm based on DRL. Then, we search the model partition point for edge-cloud collaborative inference using brute force algorithm. Fig. 4 illustrates the architecture of our CNN distributed inference system. Additionally, the algorithm of sequential and alternate task process for shortening response latency of tasks with high priorities as much as possible while reducing waiting time for tasks with low priorities is shown at the end of this section.

A. CNN Parallelism at Edge

At first, for a certain type of CNN model, we make decisions for split of each conv in it. As mentioned above, a certain group of computing resources in an edge server is only used to infer a certain conv of a CNN model and the number of resources inferring a certain conv is fixed and known. As a result, we do not need to consider the problem of resource allocation.

We adopt DRL to conduct CNN parallelism. The action space act_m is given by

$$\text{act}_m = (r_{m,1}, r_{m,2}, \dots, r_{m,A}) \quad (15)$$

where $m \in [1, M]$, and A is the total number of convs in the CNN model. The state space sta_m is given by

$$\text{sta}_m = (l_{m,1}, l_{m,2}, \dots, l_{m,A}, a_m, \Delta t_{m,1}, \Delta t_{m,2}, \dots, \Delta t_{m,A}) \quad (16)$$

where $l_{m,j} (j \in [1, A])$ is the residual height of the j th conv (we split each conv along its height). a_m is residual number of edge servers which has not got split decisions for convs. $\Delta t_{m,j}$ is the accumulation of latency difference between every two states. The meaning of $\Delta t_{m,j}$ is to make the differences of inference latencies of a conv on various edge servers as small as possible, which helps minimize the parallel inference at edge of the conv. Similarly, we convert solving the minimal parallel inference latency to solving the minimal summation of standard deviations between inference latencies on all edge servers for all convs. So the reward function is given by

$$\begin{aligned} r_m &= \sum_{j=1}^A \text{STD}(t_{1,j}, t_{2,j}, \dots, t_{M,j}) \\ &= \sum_{j=1}^A \sqrt{\frac{\sum_{m=1}^M (t_{m,j} - \frac{\sum_{v=1}^M t_{v,j}}{M})^2}{M-1}} \end{aligned} \quad (17)$$

where STD denotes the standard deviation. Note that the residual height of the j th conv $l_{m,j}$ is obtained by $l_{m,j} = l_{m-1,j} - l_{m-1,j} \cdot r_{m,j}$, and the residual number of edge servers a_m is obtained by $a_m = a_{m-1} - 1$. So it is obvious that the current

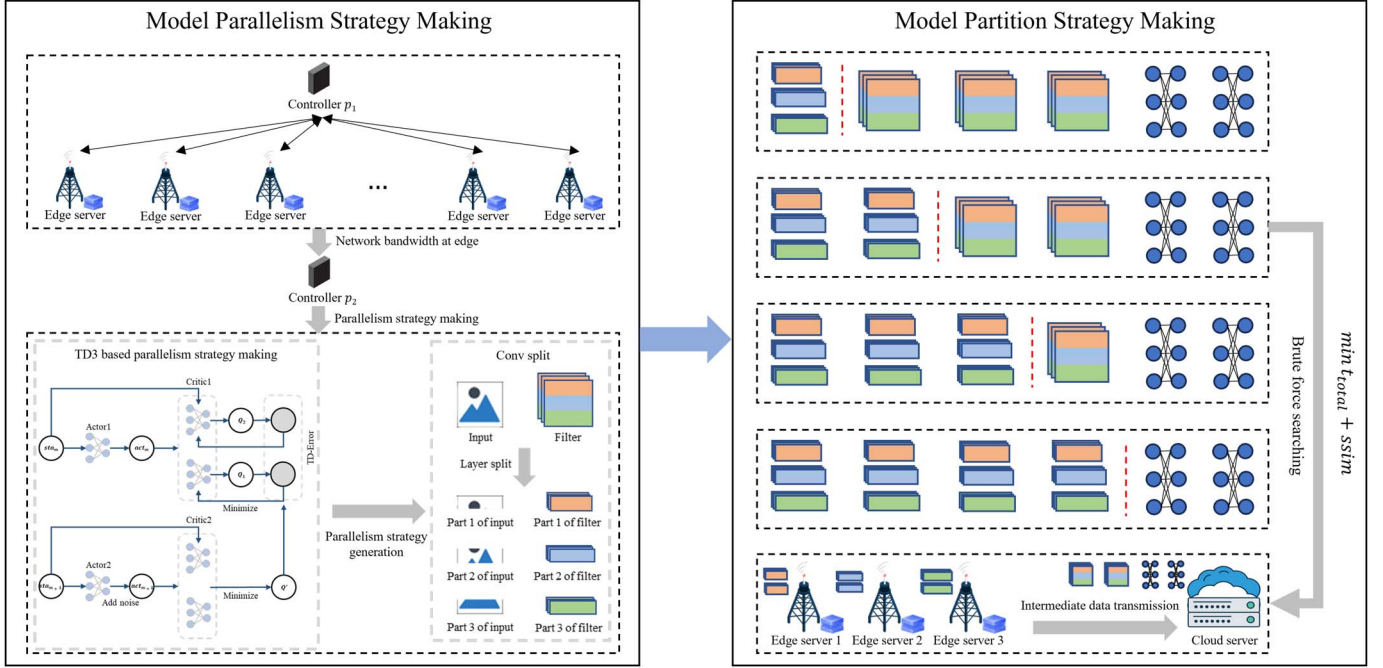


Fig. 4. Architecture of our CNN distributed inference system.

state sta_m is only dependent on the last state sta_{m-1} and the current action act_m , which means that the Markov property is met in our method.

There exist many DRL algorithms, including deep Q network (DQN) [33], [34] and deep deterministic policy gradient (DDPG) [35]. It is obvious that in our method, the value range of split rate $r_{m,j}$ is continuous, which means that the action space act_m is continuous. So DQN is not suitable here because it is only applicable to discrete action space. Although DDPG is suitable for continuous action space, it still has several drawbacks such as overestimation and low convergence speed. Focusing on shortcomings of DDPG, improvements are made on twin delayed (TD3) DDPG. Specifically, TD3 uses two critic networks and two target critic networks. For both critics and target critics, the minimal values are selected in order to alleviate the overestimation of Q. Also, delayed update of actor network is used in TD3 to improve the efficiency of learning [36]. As a result, we select TD3 to make appropriate conv split decisions considering its advantages over other DRL algorithms and its applicability in this work. Algorithm 1 shows the details of conv split algorithm.

We now explain Algorithm 1. The parameters of neural networks in TD3 are initialized randomly in lines 1–2, and an empty replay buffer is generated β in line 3. Lines 6–10 mean that the actions are set randomly in the first τ episodes of training in purpose of improving the learning efficiency of TD3. Current state, action, and reward, as well as the next state are stored into the replay buffer β in line 11. Lines 12–15 mean that we do not begin to train the critic network in TD3 until the τ th episode. When the training process of DRL finishes, the conv split decisions from the last epoch of training will be selected as the final result and be returned in line 26.

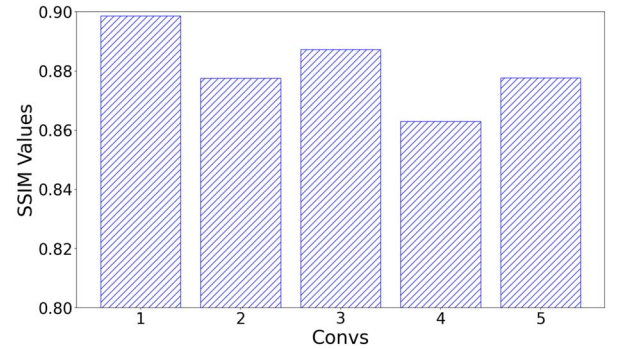


Fig. 5. Vulnerability of output from different convs.

B. CNN Partition Between Edge and Cloud

As shown in Fig. 4, the CNN partition point should be found after getting the CNN parallelism decision for further reducing the inference latency and better protecting customers' privacy.

It is emphasized that the effect of customers' privacy protection is jointly optimized with CNN inference latency during the process of CNN partition, instead of using other special and complex technologies. For a CNN, we explore the influence of different partition points on inference latency and privacy security when suffering attack.

In terms of customers' privacy protection, we use an AlexNet model with five convs to study the "vulnerability" of output from each conv. The "vulnerability" is defined by the SSIM value between customers' raw input and the data generated by attackers. The higher the SSIM value is, the more vulnerable the output from a certain conv is. As illustrated in Fig. 5, the vulnerability of output varies between different convs. It means that the risk of black-box attack can be efficiently reduced

Algorithm 1 conv split.

Input: Number of convs A , Number of edge servers M , Number of computing resources allocated to the j -th conv by the m -th edge server $k_{m,j}$, Transmission speed between the m -th edge server and controller $s_{m,p}$, Size of input for the j -th conv x_j ;

Output: Conv split decisions for a certain CNN model $\Lambda = \{\{r_{1,1}, \dots, r_{1,A}\}, \{r_{2,1}, \dots, r_{2,A}\}, \dots, \{r_{M,1}, \dots, r_{M,A}\}\}$;

- 1: Initialize critic networks Q_{θ_1} and Q_{θ_2} , and actor network π_{θ} with random parameters $\theta_1, \theta_2, \emptyset$;
- 2: Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \emptyset' \leftarrow \emptyset$;
- 3: Initialize replay buffer β ;
- 4: **for** $episode = 1$ **to** T **do**
- 5: **for** $m = 1$ **to** M **do**
- 6: **if** $episode < \tau$ **then**
- 7: $act_m = \varepsilon$;
- 8: **else**
- 9: $act_m = \pi_{\theta}(sta_m)$;
- 10: **end if**
- 11: Observe next state sta_m and reward r_m , and store transition $(sta_m, act_m, r_m, sta_{m+1})$ to β ;
- 12: **if** $episode \geq \tau$ **then**
- 13: Sample H transitions $(sta_h, act_h, r_h, sta_{h+1})_{h=1, \dots, H}$ from β ;
- 14: $y_h \leftarrow r_h + \gamma \cdot \min_{g=1,2} Q_{\theta'_g}(sta_{h+1}, \pi_{\emptyset'}(sta_{h+1}))$;
- 15: Update critics $\theta_g \leftarrow \argmin_{\theta_g} H^{-1} \cdot \sum (y_h - Q_{\theta_g}(sta_h, act_h))^2$;
- 16: **if** $episode \% v == 0$ **then**
- 17: Update $\emptyset$ by the deterministic policy gradient:
- 18: $\nabla_{\emptyset} J(\emptyset) = N^{-1} \sum \nabla_{\pi_{\emptyset}(sta_m)} Q_{\theta_1}(sta_h, \pi_{\emptyset}(sta_m)) \nabla_{\emptyset} \pi_{\emptyset}(sta_m)$;
- 19: Update target networks:
- 20: $\theta'_g \leftarrow \delta \theta_g + (1 - \delta) \theta'_g$;
- 21: $\emptyset' \leftarrow \delta \emptyset + (1 - \delta) \emptyset'$;
- 22: **end if**
- 23: **end if**
- 24: **end for**
- 25: **end for**
- 26: **return** Λ

through selecting appropriate CNN partition point. In terms of distributed inference latency of a CNN model, parallel inference of convs on several edge servers can really shorten more inference latency while bringing more data transmission delay for frequent interaction between edge devices. As a result, it may not always be a good strategy to infer all convs in parallel at edge. So we utilize the brute force algorithm to search the proper CNN partition point for further inference latency reduction while protecting customers' privacy.

C. Sequential and Alternate Task Process

The aim to shorten the service response latency of tasks with high priorities as much as possible while reducing waiting time

TABLE II
CONFIGURATION OF THE ALEXNET

Layer Number	Layer Type	Size	Stride
1	Convolution	$11 \times 11 \times 3 \times 96$	4
2	Max pooling	3×3	2
3	Convolution	$5 \times 5 \times 96 \times 256$	1
4	Max pooling	3×3	2
5	Convolution	$3 \times 3 \times 256 \times 384$	1
6	Convolution	$3 \times 3 \times 384 \times 384$	1
7	Convolution	$3 \times 3 \times 384 \times 256$	1
8	Max pooling	3×3	2
9	Fully connected	9216×4096	-
10	Fully connected	4096×4096	-
11	Fully connected	4096×10	-

for tasks with low priorities is achieved through the novel queue mechanism. Through sequential task process, tasks with higher priorities are processed earlier, which ensures the maximum reduction of response latency for tasks with high priorities. Through alternate task process, the j th conv of the n th task and the $(j+1)$ th conv of the $(n+1)$ th task can be inferred simultaneously, which means that when processing a task, the next task can also be processed without waiting the process of the current task is completely finished. This greatly decreases waiting time of tasks with low priorities. Details of sequential and alternate task process in a queue are shown in Algorithm 2.

V. EXPERIMENTAL PERFORMANCE

In this section, extensive experiments are carried out to evaluate the performance of the CNN distributed inference system proposed in this article. First, the experiment settings are described. Then, the baselines are introduced. Finally, the performance evaluation is presented.

A. Experimental Settings

For hardware facilities, the experiments are conducted on a server with Intel(R) Xeon(R) Platinum 8352V CPU and RTX 4090 GPU. Python 3.8 and PyTorch 1.11.0 are used as the simulation environment.

For setting of environmental parameters, we assume that there are ten intelligent service customers who use the same type of CNN model, eight edge servers and one cloud in a reliable CNN distributed inference system. We regard one computing resource as a thread, and the computing rate of per thread in different devices are all assumed to be 80 FLOPS. For the SMF-based connection between edge servers and the controller which is responsible for result aggregating, the transmission speed is set between 700 and 800 Gb/s [37]. While in terms of the WiFi-based connection between edge and cloud, the transmission speed is set as 3 Mb/s [38]. AlexNet [39] and CIFAR-10 [40] are used to conduct the simulation experiments. Configuration of the AlexNet used in our experiment is shown in Table II.

B. Baselines

1) *Differential Privacy*: In differential privacy [41], Gaussian noise is added to the original input from customers. In this

Algorithm 2 Sequential and alternate task process.

Input: Number of tasks in the queue N , partition point of CNN where tasks in the queue are based q ;

```

1: function INFER_CONV1(arr1,  $N$ )
2:   for  $n = 1$  to  $N$  do
3:     conv1 of the  $n$ -th task is inferred in parallel on
       edge servers;
4:     arr1[ $n$ ] = 1;
5:   end for
6: end function
7: function INFER_CONV2(arr1, arr2,  $N$ )
8:    $n = 0$ ;
9:   while True do
10:    if  $n == N$  then
11:      break;
12:    end if
13:    if arr1[ $i$ ] == 1 then
14:      conv2 of the  $n$ -th task is inferred in parallel on
        edge servers;
15:      arr2[ $n$ ] = 1;
16:       $n = n + 1$ ;
17:    end if
18:  end while
19: end function
...
20: function INFER_CONV $q$ (arr( $q - 1$ ), arr $q$ ,  $N$ )
21:    $n = 0$ ;
22:   while True do
23:    if  $n == N$  then
24:      break;
25:    end if
26:    if arr( $q - 1$ )[ $i$ ] == 1 then
27:      conv $q$  of the  $n$ -th task is inferred in parallel on
        edge servers;
28:      arr $q$ [ $n$ ] = 1;
29:       $n = n + 1$ ;
30:    if  $q == A$  then
31:      Output of conv $q$  of the  $n$ -th task is returned
        to the customer;
32:    else
33:      Output of conv $q$  of the  $n$ -th task is sent to
        the cloud;
34:    end if
35:  end while
36: end while
37: end function
...
38: arr1, arr2, ..., arr $q$  are all 1-dimension arrays with  $N$ 
   elements and all elements in them are initialized as 0;
39: Functions infer_conv1, infer_conv2, ..., infer_conv $q$  are
   run simultaneously;
40: return

```

method, both CNN parallelism and CNN partition only focus on the minimization latency, considering that privacy protection is achieved through the utilization of noise. In order to select

TABLE III
SELECTION OF STANDARD DEVIATION FOR
GAUSSIAN NOISE

Std	Reduction of Accuracy	SSIM Values
0.20	4.55%	0.8663
0.21	13.45%	0.8650
0.22	42.41%	0.8637
0.23	68.18%	0.8626

a proper value for the standard deviation of Gaussian noise and for simplicity, we choose several values in advance, among which the most proper one may exist. Then, for each value chosen by us, the reduction of accuracy and the vulnerability of input from the last conv (i.e., we assume all convs are inferred at edge here) are observed to decide the most proper standard deviation for Gaussian noise. As shown in Table III, with the increasing of the standard deviation value of Gaussian noise, accuracy and security of the final output, respectively, reduces and improves. Comprehensively considering the influence of different standard deviations of Gaussian noise on final output's accuracy and SSIM, we choose 0.21 as the standard deviation of Gaussian noise because of the relatively acceptable accuracy loss and lower SSIM of output compared with setting the standard deviation of Gaussian noise as 0.2.

2) *Edge-Cloud Partition*: In this method, CNN parallelism is not used and privacy protection is not taken into account. The CNN model is only partitioned into two parts, one part inferred at edge while the other one inferred on the cloud, with the goal to minimize the inference latency.

C. Performance Evaluation

1) *Performance on Inference Latency*: We first consider inference latency of the intelligent service with the highest priority in distributed inference system. We compare the performance of our method with two other ones. In detail, it should be emphasized that in edge-cloud Partition, we assume that all services are processed in parallel without any queuing. In addition, note that the number of threads allocated to the service is only possible to change with the total request number changing in edge-cloud partition, i.e., the number of threads allocated to each service will not change in our method and differential privacy. As shown in Fig. 6, the performance of differential privacy is the best, followed by our method and edge-cloud partition. The inference latency of differential privacy is about 0.3 ms lower than our method. The reason is that customers' privacy is protected through directly adding noise to pictures, and the decisions of conv split and CNN partition point are only focused on reduction of latency in differential privacy, while inference latency and security of customers' privacy are joint optimized during the process of CNN partition in our method. It is also obvious that inference latency of edge-cloud partition is much larger than the other two methods when there are too many customers in the distributed inference system and the number of computing resources allocated to each customer is so small. Specifically, when only one thread is allocated to process the service, the inference latency of edge-cloud partition is almost three times larger than the other two methods,

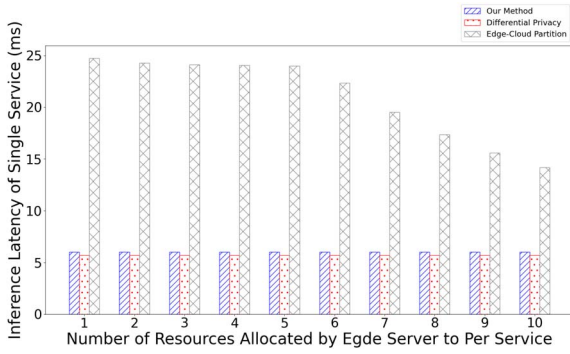


Fig. 6. Inference latency of a single intelligent service with different numbers of computing resources allocated.

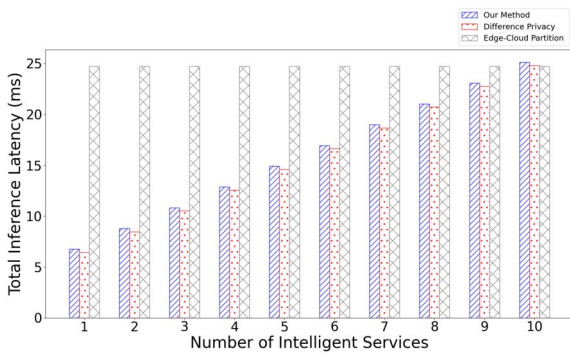


Fig. 7. Total response time of various numbers of intelligent services.

and the gap decreases by about one time when ten threads are allocated to the service. It proves the effect of the improved queue mechanism and CNN parallelism in our method.

We then consider the inference latency of each intelligent services in the distributed inference system with ultrahigh workload (i.e., each task is only allocated to one thread in edge–cloud partition only), which includes the queuing time. As shown in Fig. 7, the inference latency of our method is always a little larger than difference privacy (only about 2 ms), which can be neglected in most conditions. However, the inference latency for edge–cloud partition is much larger than other two methods until for the tenth service. Specifically, the inference latency of edge–cloud partition is almost three times as large as those of other two methods for the first service. The gap keeps becoming smaller, and inference latencies of our method and difference privacy begin to exceed that of edge–cloud partition from the tenth service in the queue. This observation proves that our method is suitable to services with high priorities in a distributed inference system with so many customers.

2) *Performance on Privacy Protection:* As illustrated in Fig. 8, the vulnerability of output from different convs of a CNN model can be various. However, the Gaussian noise added to the original input does not have serious influence on the vulnerability of output from most convs (three convs in the middle as shown here). The average SSIM values of generated pictures from the second, third, and fourth conv's output only reduce by about 0.005, 0.003, and 0.001, respectively, which are

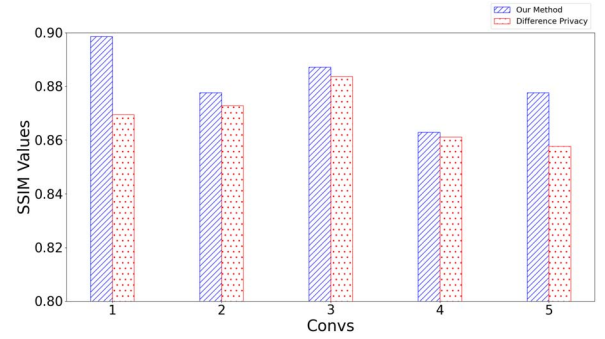


Fig. 8. Comparison of performance on privacy protection between two methods.

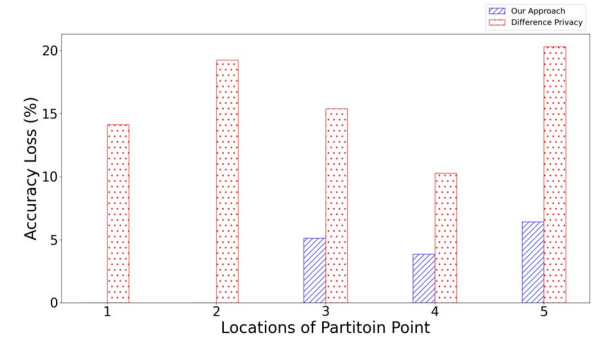


Fig. 9. Accuracy loss compared to original network.

negligible. Note that, in difference privacy, the partition point of CNN is selected without considering privacy protection, which means that the found point in difference privacy is a relatively bad choice for privacy protection. Different from difference privacy, our method fully utilizes the differences between vulnerability of output from various convs when conducting CNN partition. Therefore, our method can perform much better than single difference privacy on privacy protection for customers in general.

3) *Performance on Accuracy Loss:* Fig. 9 illustrates the accuracy loss of our method and difference privacy compared to that of original network, where the convs are not split and the input data are not added with Gaussian noise. It is obvious that the accuracy loss of our method is so small that can be overlooked, while that of difference privacy is much larger. Specifically, the accuracy loss of our method in all possible model partition conditions is lower than 10%, while that of difference privacy in all possible model partition conditions exceeds 10% and the largest loss even reaches 20%, which significantly reduces QoS. Through this experiment, we can get that our method well ensures the quality of returned results of intelligent services while difference privacy does not.

VI. CONCLUSION

In this article, we design a novel CNN distributed inference method to reduce the response time of intelligent services while protecting customers' privacy as much as possible. This distributed inference method includes two key problems, which

are CNN parallelism and CNN partition. The performance of our method is compared with two other methods in inference latency, performance on privacy protection and accuracy loss through simulation experiments. The results of simulation show that the method proposed in this article performs better than the compared methods and is applicable for intelligent services with the highest priorities in the distributed inference system. However, because we mainly focus on the distributed inference method of CNNs, the network condition is assumed to be so good at edge. In reality, data transmission speed is often lower than that set in this article, so the research on data transmission latency reduction in model parallelism is worth conducting in the future.

REFERENCES

- [1] J. Redmon and A. Farhadi, "Yolo9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7263–7271.
- [2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, vol. 28.
- [3] X. Hou, Y. Guan, T. Han, and N. Zhang, "DistrEdge: Speeding up convolutional neural network inference on distributed edge devices," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 1097–1107.
- [4] X. Xu, J. Gu, H. Yan, W. Liu, L. Qi, and X. Zhou, "Reputation-aware supplier assessment for blockchain-enabled supply chain in industry 4.0," *IEEE Trans. Ind. Inform.*, vol. 19, no. 4, pp. 5485–5494, Apr. 2023.
- [5] S. Chen, Q. Li, M. Zhou, and A. Abusorrah, "Recent advances in collaborative scheduling of computing tasks in an edge computing paradigm," *Sensors*, vol. 21, no. 3, 2021, Art. no. 779.
- [6] X. Hou and T. Han, "TrustServing: A quality inspection sampling approach for remote DNN services," in *Proc. 17th Annu. IEEE Int. Conf. Sens., Commun., Netw. (SECON)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1–9, 2020.
- [7] X. Xu, Z. Liu, M. Bilal, S. Vimal, and H. Song, "Computation offloading and service caching for intelligent transportation systems with digital twin," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20757–20772, Nov. 2022.
- [8] Z. Xia, Y. Zhu, X. Sun, Z. Qin, and K. Ren, "Towards privacy-preserving content-based image retrieval in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 6, no. 1, pp. 276–286, Jan.–Mar. 2018.
- [9] X. Xu, S. Tang, L. Qi, X. Zhou, F. Dai, and W. Dou, "CNN partitioning and offloading for vehicular edge networks in Web3," *IEEE Commun. Mag.*, vol. 61, no. 8, pp. 36–42, Aug. 2023.
- [10] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Comput. Archit. News*, vol. 45, no. 1, pp. 615–629, 2017.
- [11] Z. Li, M. Bilal, X. Xu, J. Jiang, and Y. Cui, "Federated learning-based cross-enterprise recommendation with graph neural networks," *IEEE Trans. Ind. Inform.*, vol. 19, no. 1, pp. 673–682, Jan. 2023.
- [12] B. Shneiderman, "Bridging the gap between ethics and practice: Guidelines for reliable, safe, and trustworthy human-centered AI systems," *ACM Trans. Interactive Intell. Syst.*, vol. 10, no. 4, pp. 1–31, 2020.
- [13] Z. He, T. Zhang, and R. Lee, "Attacking and protecting data privacy in edge-cloud collaborative inference systems," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9706–9716, Jun. 2021.
- [14] L. Pengcheng, J. Yi, and L. Zhang, "Query-efficient black-box attack by active learning," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 1200–1205.
- [15] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive DNN surgery for inference acceleration on the edge," in *Proc. IEEE Conf. Comput. Commun.*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 1423–1431.
- [16] Y. Boykov and V. Kolmogorov, "An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 26, no. 9, pp. 1124–1137, Sep. 2004.
- [17] Y. Duan and J. Wu, "Joint optimization of DNN partition and scheduling for mobile cloud computing," in *Proc. 50th Int. Conf. Parallel Process.*, 2021, pp. 1–10.
- [18] S. Johnson, "Optimal two- and three-stage production schedules with setup times included," *Naval Res. Logistics Quart.*, vol. 1, no. 1, pp. 61–68, Mar. 1954.
- [19] J. Li, W. Liang, Y. Li, Z. Xu, X. Jia, and S. Guo, "Throughput maximization of delay-aware DNN inference in edge computing by exploring DNN model partitioning and inference parallelism," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 3017–3030, May 2023.
- [20] J. Kim, Y. Park, G. Kim, and S. Hwang, "SplitNet: Learning to semantically split deep networks for parameter reduction and model parallelization," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 1866–1874.
- [21] S. Tuli, G. Casale, and N. Jennings, "SplitPlace: AI augmented splitting and placement of large-scale neural networks in mobile edge environments," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5539–5554, Sep. 2023.
- [22] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Co-operative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, Apr. 2020.
- [23] P. Kairouz, Z. Liu, and T. Steinke, "The distributed discrete Gaussian mechanism for federated learning with secure aggregation," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 5201–5212.
- [24] L. Zhao, J. Jiang, B. Feng, Q. Wang, C. Shen, and Q. Li, "SEAR: Secure and efficient aggregation for byzantine-robust federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3329–3342, Sep./Oct. 2022.
- [25] S. Xie, B. Liu, and Y. Hong, "Privacy-preserving cloud-based DNN inference," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 2675–2679.
- [26] Y. Tian, L. Njilla, J. Yuan, and S. Yu, "Low-latency privacy-preserving outsourcing of deep neural network inference," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3300–3309, Mar. 2021.
- [27] Y. Huang, F. Wang, F. Wang, and J. Liu, "DeePar: A hybrid device-edge-cloud execution framework for mobile deep learning applications," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 892–897.
- [28] A. Eshratifar, M. Abrishami, and M. Pedram, "JointDNN: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 565–576, Feb. 2021.
- [29] J. Mendiñeta, S. Shinada, Y. Hirota, H. Furukawa, and N. Wada, "High-capacity super-channel-enabled multi-core fiber optical switching system for converged inter/intra data center and edge optical networks," *IEEE J. Sel. Topics in Quantum Electron.*, vol. 26, no. 4, pp. 1–13, Jul./Aug. 2020.
- [30] L. Guegan and A.-C. Orgerie, "Estimating the end-to-end energy consumption of low-bandwidth IoT applications for WiFi devices," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 287–294.
- [31] Y. Wen, J. Liu, W. Dou, X. Xu, B. Cao, and J. Chen, "Scheduling workflows with privacy protection constraints for big data applications on cloud," *Future Gener. Comput. Syst.*, vol. 108, pp. 1084–1091, Jul. 2020.
- [32] J. Wang, P. Chen, N. Zheng, B. Chen, J. Principe, and F.-Y. Wang, "Associations between MSE and SSIM as cost functions in linear decomposition with application to bit allocation for sparse coding," *Neurocomputing*, vol. 422, pp. 139–149, Jan. 2021.
- [33] Y. Xu, Y. Wei, K. Jiang, L. Chen, D. Wang, and H. Deng, "Action decoupled SAC reinforcement learning with discrete-continuous hybrid action spaces," *Neurocomputing*, vol. 537, pp. 141–151, Jun. 2023.
- [34] C. Yang, X. Xu, X. Zhou, and L. Qi, "Deep Q network-driven task offloading for efficient multimedia data analysis in edge computing-assisted IoV," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 18, no. 2s, pp. 1–24, 2022.
- [35] Y. Yu, J. Tang, J. Huang, X. Zhang, D. So, and K.-K. Wong, "Multi-objective optimization for UAV-assisted wireless powered IoT networks based on extended DDPG algorithm," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 6361–6374, Sep. 2021.
- [36] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. 35th Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 1587–1596.
- [37] H. Zhang et al., "800 Gbit/s transmission over 1 km singlemode fiber using a four-channel silicon photonic transmitter," *Photon. Res.*, vol. 8, no. 11, pp. 1776–1782, 2020.

- [38] J. Lee, Y. Kim, S. Moon, and J. Ko, "A reconfigurable neural architecture for edge-cloud collaborative real-time object detection," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 23390–23404, Dec. 2022.
- [39] F. Iandola, S. Han, M. Moskewicz, K. Ashraf, W. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and 0.5 mb model size," 2016, *arXiv:1602.07360*.
- [40] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," 2009.
- [41] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends® Theor. Comput. Sci.*, vol. 9, nos. 3–4, pp. 211–407, 2014.



Yuhao Hu received the B.S. degree in mechanical engineering from Nanjing Tech University, Nanjing, China, in 2021. He is currently working toward the master's degree with the School of Software Engineering, Nanjing University of Information Science and Technology, Nanjing, China.

His research interests include mobile edge computing, big data, Internet of Things, and deep learning.



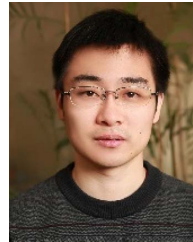
Xiaolong Xu (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from Nanjing University, Nanjing, China, in 2016.

He was a Research Scholar with Michigan State University, East Lansing, Michigan, USA, from April 2017 to May 2018. He is currently a Full Professor with the School of Software Engineering, Nanjing University of Information Science and Technology, Nanjing, China. He has published more than 100 peer-review articles in international journals and conferences, including *IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS*, *IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS*, *IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS*, *ACM Transactions On Sensor Networks*, *ACM Transactions On Multimedia Computing, Communications, and Applications*, *ACM Transactions On Intelligent Systems and Technology*, *IEEE ICWS*, and *ICSOC*. He was selected as the Highly Cited Researcher of Clarivate 2021 and 2022. His research interests include big data, Internet of Things, and deep learning.



Lianyong Qi (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from the Department of Computer Science and Technology, Nanjing University, Nanjing, China, in 2011.

He is currently a Full Professor with the College of Computer Science and Technology, China University of Petroleum (East China), Qingdao, China. He has already published more than 50 articles, including *IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS*, *IEEE TRANSACTIONS ON CLOUD COMPUTING*, *IEEE TRANSACTIONS ON BIG DATA*, *Future Generation Computer Systems*, *Journal of Computational Social Science*, *CCPE*, *ICWS*, and *ICSOC*. His research interests include services computing, big data, and Internet of Things.



Xiaokang Zhou (Member, IEEE) received the Ph.D. degree in human sciences from Waseda University, Tokyo, Japan, in 2014.

He is currently an Associate Professor with the Faculty of Data Science, Shiga University, Hikone, Japan. From 2012 to 2015, he was a Research Associate with the Faculty of Human Sciences, Waseda University, Tokyo, Japan. He has been working as a Visiting Researcher with the RIKEN Center for Advanced Intelligence Project (AIP), RIKEN, Tokyo, Japan, since 2017. He has been engaged in interdisciplinary research works in the fields of computer science and engineering, information systems, and social and human informatics. His recent research interests include ubiquitous computing, big data, machine learning, behavior and cognitive informatics, cyber-physical-social systems, and cyber intelligence and security.

Dr. Zhou is a member of ACM, USA; IPSJ and JSAI, Japan; and CCF, China.



Xiaoyu Xia received the Ph.D. degree in computer science and technology from Deakin University, Melbourne, VIC, Australia in 2023.

Currently, he is a Lecturer with RMIT University, Melbourne, VIC, Australia. His research interests include edge computing, service computing, and system security. More details about his research can be found at <https://sites.google.com/view/xiaoyuxia/>.